

Jira and Xray Testing Workflow

A working guide for the IV&V testing team

Version 1.0 | September 25 2026

Use this guide to turn an email or meeting notification into a traceable testing assignment, select reusable test cases, record results, and report defects. Each tester manages the work for their assigned application in our shared Jira project.

Applications are tracked with Components and releases with Fix Version/s. Use one Test Plan per application and release; its scope can include regression, UI, automation, performance, accessibility, or other requested testing.

The workflow at a glance

Receive the testing window → create a Task → confirm Component and Fix Version → create or reuse the Test Plan → select Tests → create Test Executions → record results and Bugs → retest and report completion.

Item	What it represents
Task	The testing assignment, owner, window, and completion summary. Creating a Task is a recommended addition to our process.
Test Repository and Test	The reusable test library and individual test case definitions. A Test can be used in many releases.
Test Plan	The agreed scope for one application release, with selected Tests and related executions.
Test Execution	A defined testing round for a particular context, such as browser, build, environment, or testing category.
Test Run	One Test instance within a Test Execution. This is where execution results and evidence belong.
Bug	A defect linked to the failed run or step so another person can reproduce the failure.

How to use this guide

The steps describe the team process; button names, fields, and Jira statuses can differ in Maestro. Use the equivalent controls in your project. All screen illustrations are examples to replace with actual screenshots. Sample names and issue keys are fictional. Product references are listed in section 9.

1 Receive and register the assignment

When the testing window is announced, record enough detail for another tester to understand the work and pick it up if needed.

- 1. Confirm the application, release or build, window start and end with time zone, testing scope, environment, and requesting contact. Record open questions before execution.
- 2. Select Create in Jira and choose Task in the shared IV&V project. Use a summary such as “APP01 | 2026.09 | IVV testing | September window”. Assign it to the responsible tester.
- 3. Set Component to the application. In the description, capture the request source, scope, dates, access needs, environment, and expected deliverables. Link the notification or summarize the meeting decision.
- 4. Check the project Releases or Versions area for the correct application release. Reuse it if present; otherwise create it if permitted, or ask the person who manages versions. Set Fix Version/s on the Task.
- 5. Link the Test Plan when created. Update the Task as scope or dates change, using the project’s existing status workflow.

Keep releases distinct across applications

Versions belong to the shared Jira project, not to an individual Component. Recommended naming is “APP01-2026.09” or the established application code plus release number. Avoid a generic “1.0” if several applications use that number. The testing window is separate from the release date; record both when known. [1]

ILLUSTRATIVE SCREEN | Replace with your Maestro Jira screenshot

IVV project

Work items

Releases

Testing

Repository

Create Task

Project / Type

Summary

Component

Fix Version/s

Assignee / Window

IVV / Task

APP01 | 2026.09 | IVV testing

APP01 Claims Portal

APP01-2026.09

Assigned tester / dates and time zone

Create Task

Figure 1. Replace with the Task creation screen showing Component, Fix Version/s, assignee, and testing dates.

2 Create the release Test Plan

Search for an existing Test Plan for the same application and release before creating another. Reuse that plan for an additional window in the same release, updating its scope and adding new executions. Create a new plan for a new release.

1. Choose Create and select Test Plan. Name it “APP01 | 2026.09 | IVV Test Plan”.
2. Set the application Component, release Fix Version/s, and responsible tester. Link the assignment Task with the available Jira issue link.
3. Complete the description using the checklist below. Add only the testing categories requested for this release; a Test Plan does not need a different issue type for each category.
4. Save the plan, add the selected Tests, and later associate each Test Execution with it through Xray. A generic Jira link alone is not the Xray Test Plan association.

Include	What to record
Purpose and scope	What changed, what must be validated, references or requirements, and explicit exclusions.
Testing categories	Regression, manual UI, automated UI, performance, Section 508 accessibility, or another named category.
Execution context	Environment and URL, build, browsers and versions, operating system, tools, accounts, and test data.
Window and ownership	Start and end with time zone, tester, requester, dependencies, and known constraints.
Readiness	Accessible build, available accounts/data, reviewed tests, and agreed expected results or thresholds.
Completion and evidence	Recorded results, screenshot at each failed manual step, linked Bugs, explained unexecuted work, and a completion summary.
Selected Tests	Links to the reusable Tests that cover the agreed scope. Record coverage gaps and new tests needed.

Choose scope without changing Xray test definitions unnecessarily

“Performance”, “508”, and “regression” describe testing scope. Selenium, Playwright, LRE, and JMeter are tools; Edge and Firefox are browsers. They are not automatically Xray Test Type values. Capture these details in the description and existing fields or labels. Use the configured Test Type appropriate to the reusable test definition.

3 Select tests from the repository

The Test Repository organizes reusable test cases. Adding an existing Test to a plan or execution reuses its definition; it does not require a cloned Test for every release. [2]

1. Open the project's Xray testing area and Test Repository, or use Jira search for Test issues. Filter to the correct application Component and relevant folders or labels.
2. Review each candidate Test: objective, preconditions, test data, steps, expected results, and relevance to the release. Check browser or environment limitations.
3. In the Test Plan Tests area, use Add Tests or the equivalent selection action. Select existing Test keys from the repository or search results and add them to the plan.
4. Review the final test list against the requested scope. If coverage is missing, create or update a Test using the team's existing review practice. For a manual Test, provide an action, data where needed, and expected result for each step.
5. Keep reusable test definitions independent of a single execution. Track the tested release on the plan and executions. Do not overwrite shared Tests just to record this release's results.

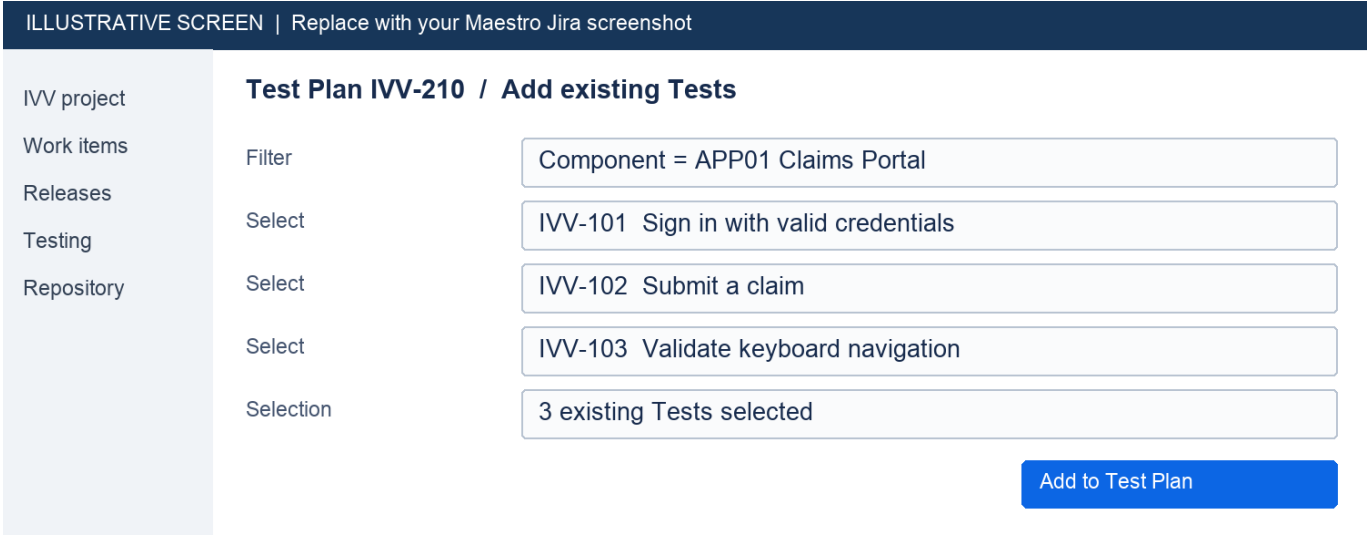


Figure 2. Replace with the repository or Add Tests selection view showing application filtering and selected Tests.

If a test is missing

Search by its issue key and title, check filters and permissions, and confirm whether the Test itself or only an old Test Run was archived. Follow section 7 before recreating a case that may already exist.

4 Create and organize Test Executions

Create an execution for a specific testing context. The same Test may run in several executions while remaining a single reusable Test. Use separate executions when you need independent results for Edge and Firefox, different builds, or a retest round. [3]

1. From the Test Plan, select the relevant Tests and use the available Create Test Execution action. Alternatively create a Test Execution, associate the Test Plan using Xray's Test Plans field, and add the selected Tests.
2. Name it clearly, for example "APP01 | 2026.09 | UI regression | Edge | Round 1". Set Component, Fix Version/s, and assignee; verify these even when they appear to be copied from the plan.
3. Record the environment, build, browser/version, tool, dates, and test data. Use Test Environments if configured; otherwise record the context in the description. Do not combine browser results into one ambiguous outcome.
4. Verify the execution's Tests are in the plan scope and assigned to the appropriate tester. Start each Test Run from this execution when testing begins.

ILLUSTRATIVE SCREEN | Replace with your Maestro Jira screenshot

IVV project	Test Execution IVV-220	
Work items	Test Plan	IVV-210 APP01 release Test Plan
Releases	Component / Version	APP01 Claims Portal / APP01-2026.09
Testing	Context	UAT / build 2026.09.3 / Edge 140
Repository	Scope / Round	UI regression / Round 1
	Tests	IVV-101, IVV-102 Not started
		Open Test Run

Figure 3. Replace with a Test Execution showing its Xray Test Plan association, context, and Tests.

Examples within one release plan

An application release may have an Edge regression execution, a Firefox regression execution, an LRE performance execution, and a Section 508 execution. Add only those requested. A later fix can have a separate "Retest | build 2026.09.4" execution under the same plan.

5 Execute tests and record results

Our team records results manually in Xray, including results from Selenium, Playwright, LRE, and JMeter runs. Running a tool does not automatically update Xray in this process.

- 1. Open the Test Run from the correct Test Execution. Confirm the application, release, build, and test context before starting.
- 2. For a manual Test, perform the steps and compare actual behavior with each expected result. Mark the completed steps using the available pass or fail controls.
- 3. When a step fails, mark it failed, describe the actual result, and attach the screenshot to that step's evidence area. Include enough context to identify the failure and protect sensitive data.
- 4. Create or link the Bug at the failed step or Test Run. Save the changes and verify the overall run result reflects the work performed. Step evidence and defect controls depend on the Test Type and configuration. [4]
- 5. If testing cannot proceed, record the reason and use the configured blocked, pending, or not-run state. Do not mark unexecuted work as passed. Jira issue status and Xray run result are separate: closing an issue does not prove a test passed.

ILLUSTRATIVE SCREEN | Replace with your Maestro Jira screenshot

IVV project	Run of Test IVV-102 / Execution IVV-220	
Work items	Step 3	Submit the completed claim
Releases	Expected	Confirmation number is displayed
Testing	Actual / Result	Error displayed / FAIL
Repository	Step evidence	IVV-102_step3_build2026.09.3.png
	Linked defect	IVV-230 Submission fails

Save result

Figure 4. Replace with the manual execution screen showing a failed step, its screenshot attachment, and linked Bug.

Results from tools and tests without steps

Review the tool report before entering pass or fail. Attach the relevant report, log, trace, or recording to the Test Run and identify the scenario, tool run, and build. If the Test has no manual steps, attach evidence at run level and identify the failing check in the comment. For performance, compare measured results with the agreed thresholds; for accessibility, record the agreed criterion and observed behavior.

6 Report defects and retest fixes

A failed test needs enough information for another person to reproduce the problem. Check for an existing Bug first; link it when it describes the same defect.

1. From the failed step or run, create a Bug using the available defect action, or create it in Jira and link it back through Xray's defect control.
2. Use a specific summary such as "APP01 | Submit claim | Error instead of confirmation". Record the application, tested release/build, environment, browser, prerequisites, and steps to reproduce.
3. Include expected and actual results, screenshot or report, impact, and Test/Execution keys. Set severity and priority using the fields and conventions available in the project.
4. Record the observed release in Affects Version/s if used, and always in the defect description. On a Bug, Fix Version/s normally means the planned fix release; do not assume it equals the release tested.
5. After a fix is delivered, create a new retest execution for the relevant Tests under the same release plan when appropriate. Record the new build and link the original Bug. Preserve the earlier failed run and its evidence.
6. Run the affected checks again. Update the Bug with the retest result and use the team's existing defect transitions. If it still fails, keep or reopen the Bug as allowed and attach the new evidence.

Record the end of the testing window

Review every execution in scope, including each requested browser and testing category. A consolidated latest result can hide differences between rounds or environments; verify the individual execution results before reporting completion.

- Every planned Test has a result or an explicit reason it was not executed.
- Failed manual steps have screenshots and linked Bugs; tool-based failures have supporting evidence.
- Retest results are recorded and remaining defects, blockers, and exclusions are listed.
- The Task and Test Plan link to the relevant executions and a concise completion summary.

Share the outcome with the requesting contact through the normal team channel. Move the Task and other issues through the existing Jira workflow. Completion of testing does not mean every test passed or that IV&V has authorized the application release.

7 Locate archived tests and results

Treat a missing Test as a lookup problem first. Maestro may apply local archive rules; the applicable rule and restoration permissions must be confirmed with the Maestro/Jira support team.

Distinguish the archived item

Item	Meaning and action
Test issue	The reusable case may be unavailable for selection. Ask support to locate the original Test and restore it if supported and appropriate.
Test Run	A historical execution result may be hidden while the reusable Test remains available. Xray Cloud run archiving preserves run data and allows archived results to be browsed. [5]
Project or access	The project may be archived, or your permissions may have changed. Ask support to identify the cause before requesting a new Test.

1. Search by Test key and title; remove restrictive filters and check the application Component and repository folder.
2. If available, inspect execution history or the view for archived Test Runs. An archived result alone does not mean the reusable Test needs restoration.
3. Send the support team the Test key or old link, application, reason it is needed, target release, and testing window. Include the archive notification if available.
4. Ask an authorized administrator or support owner to confirm what was archived and the permitted restoration or access process. An administrator role alone is not a guarantee that every archived or deleted item can be restored.
5. After access or restoration, review the Test for current behavior, data, and prerequisites. Add it to the current plan and create a fresh execution. Keep historical evidence associated with its original run.

Suggested restoration request

Please help locate Test [key or link] for [application]. We need it for [release] during [testing window]. It is missing from [repository/search/execution view]. Please confirm whether the Test issue, a Test Run, or the project is archived and restore access where supported. Related notification or prior execution: [link].

Avoid creating a duplicate solely because a Test is missing. If the original cannot be recovered, record that outcome in the Task and create a replacement with a reference to the old key. Do not delete test history to resolve an archive issue.

8 Copyable descriptions and completion checklist

Task description

Application and Component: [name]
Release and build: [version / build]
Testing window and time zone: [start / end]
Request source and contact: [notification or meeting / contact]
Requested scope: [testing categories and features]
Environment and access: [URL / accounts / data]
Assigned tester: [name]
Test Plan: [key]
Constraints and open questions: [details]

Test Plan description

Objective: [what this release needs to validate]
In scope: [features and testing categories]
Out of scope: [exclusions]
Reference material: [requirements / change notes]
Application and release: [Component / Fix Version]
Context: [environment / build / browsers / tools]
Window and owner: [dates, time zone / tester]
Prerequisites and data: [readiness needs]
Expected results or thresholds: [criteria]
Test selection: [Test keys / coverage gaps]
Execution breakdown: [category / browser / round]
Completion evidence: [results / failed-step screenshots / Bugs / summary]

Completion summary

Application and release: [name / version]
Task and Test Plan: [keys]
Window completed: [date / time zone]
Executions reviewed: [keys / builds / browsers / categories]
Results by execution: [pass / fail / blocked / not run]
Open defects and retest outcomes: [keys / details]
Scope not completed and reason: [details]
Evidence and reports: [links]
Next action or handoff: [owner / action]

Before you send the summary

Confirm the application and release are correct, executions are associated with the plan, required evidence is attached to failures, and the summary matches the recorded results. When reporting totals, state whether you counted unique Tests or Test Runs; the same Test executed in two browsers represents two runs.

9 Replace illustrations and maintain the guide

Replace the sample screens

Each illustration is a single image. In Word, select it and use Change Picture to insert the corresponding Maestro screenshot. Keep it inline with text, preserve its proportions, and update the caption. Remove the sample-screen wording when a real screenshot replaces it. Use representative data and conceal sensitive information.

Figure	Capture from your Jira
1	Task creation with Component, Fix Version/s, assignee, and testing window.
2	Repository or Add Tests view with the application filter and selected Tests.
3	Test Execution with associated Test Plan, execution context, and test list.
4	Failed manual step with evidence attachment and linked Bug.

Confirm local configuration before team publication

The guide owner should confirm the Maestro deployment and Xray edition, visible field and action names, permitted status transitions, version naming, and support contact for archived items. Confirm whether the team needs a formal review or signoff step. The Task tracking and naming conventions in this guide are proposed working practices, not additional Xray requirements.

Official product references

These references explain the product concepts behind the guide. Local Maestro configuration and team procedures determine the exact screens and permissions. Documentation checked September 25 2026.

[1] [Atlassian Create a version](#)

[2] [Xray Cloud Getting Started](#)

[3] [Xray Cloud Test Execution](#)

[4] [Xray Cloud Execute Tests](#)

[5] [Xray Cloud Test Run Archiving](#)